Contents

The course will mainly cover the following topics:

- ✓ A Gentle Introduction to Machine Learning
- ✓ Linear Regression
- ✓ Logistic Regression
- ✓ Naive Bayes
- ✓ Support Vector Machines
- ✓ Decision Trees and Ensemble Learning
- ✓ Clustering Fundamentals
- ✓ Hierarchical Clustering
- ✓ Neural Networks and Deep Learning
- ✓ Unsupervised Learning……

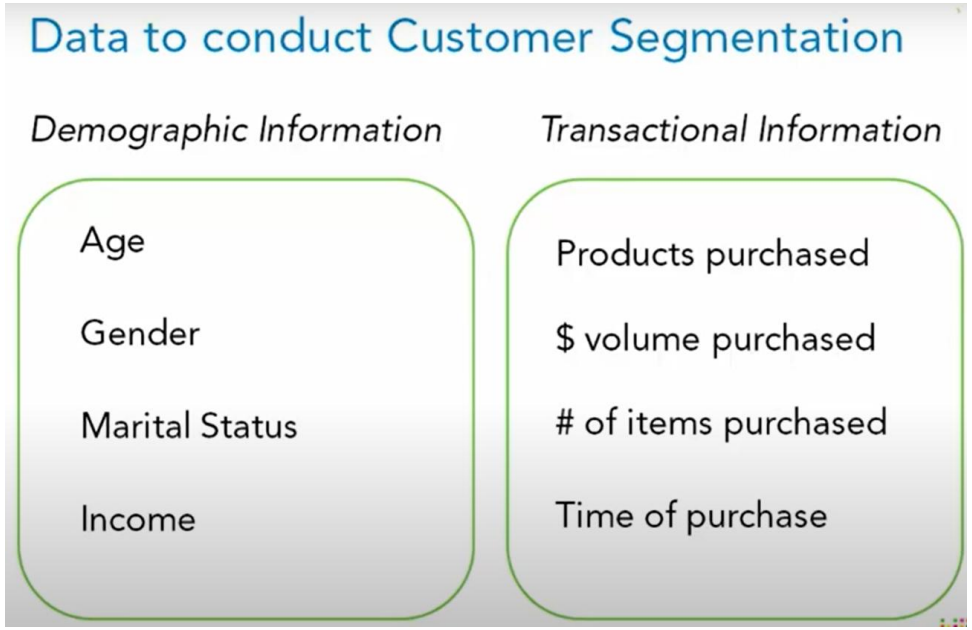
Outline

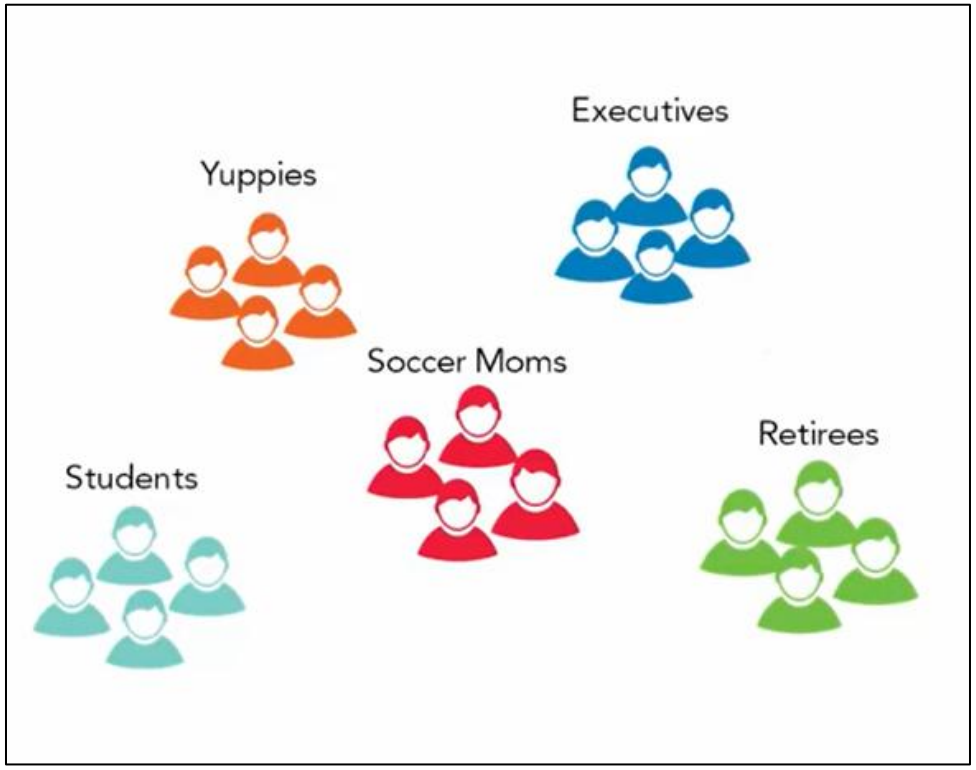
- ✓ A Gentle Introduction to Machine Learning
  - Unsupervised learning
  - Reinforcement learning
- ✓ Linear Regression
  - Linear models
  - **Hypothesis function for Linear Regression**
  - **Cost function**
  - Gradient descent algorithm
  - Polynomial regression

Unsupervised machine learning

- ✓ **Unsupervised learning** is a type of machine learning where the algorithm learns from **unlabeled data** without any predefined outputs or target variables.
- ✓ It finds **hidden patterns, similarities, or groupings** within the data to get insights and make data-driven decisions without the need for human intervention..
- ✓ It is particularly **useful when dealing with large datasets** where **manual labeling would be impractical or costly**.
- ✓ This method's ability to discover similarities and differences in information make it ideal for **exploratory data analysis, cross-selling strategies, customer segmentation, and image and pattern recognition**.
  - Cross-selling is the process of offering a customer products that are compatible with the ones they're purchasing

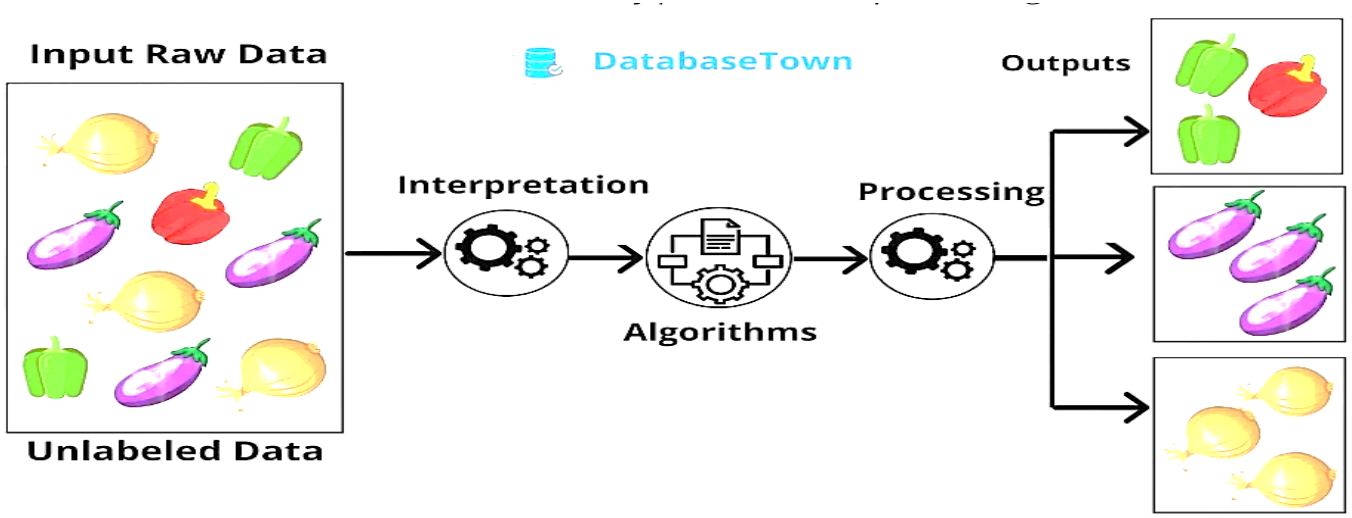
✓ Customer segmentation is the process by which you divide your customers based on common characteristics – such as demographics or behaviours, so your marketing team or sales team can reach out to those customers more effectively.





How does Unsupervised Learning Work?

- ✓ Suppose an input dataset containing images of different types of cats and dogs. Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are not given.
- ✓ Now, this unlabeled input data is fed to the machine learning model in order to train it.
- ✓ Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as K-means clustering algorithms.
- ✓ Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.



Types of Unsupervised Learning

- ✓ **Unsupervised** learning can be broken down into three main tasks:
  - Clustering
  - Association rules
  - Dimensionality reduction.

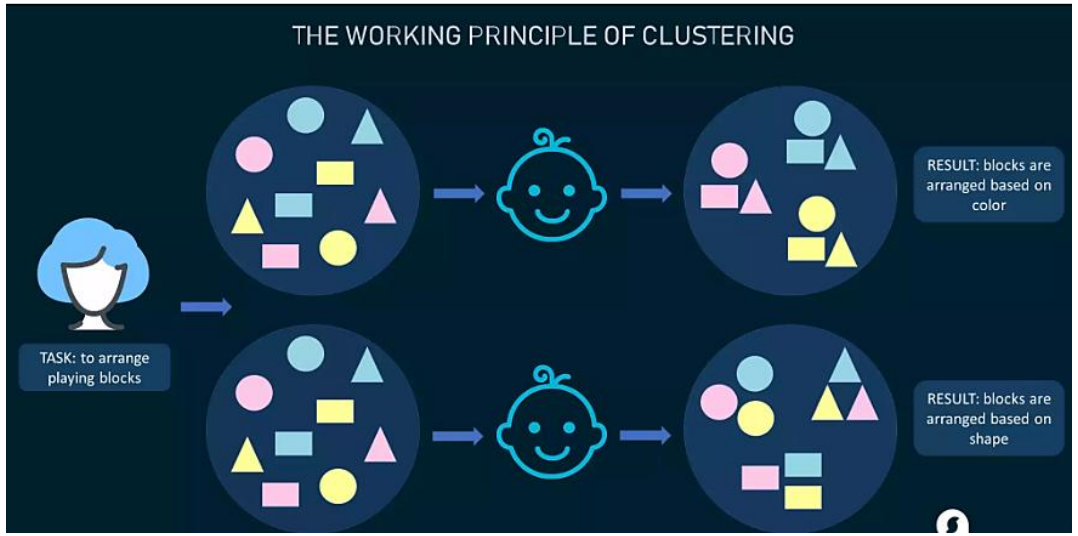
Unsupervised Algorithm:

1. K-means clustering
2. KNN (k-nearest neighbours)
3. Hierarchal clustering
4. Anomaly detection
5. Neural Networks
6. Principle Component Analysis
7. Apriori algorithm

Types of Unsupervised Learning

Clustering Algorithms

✓ Clustering algorithms only interpret the input data and find natural groups or clusters in feature space



✓ Clustering is a popular type of unsupervised learning approach. You can even break it down further into different types of clustering; for example:

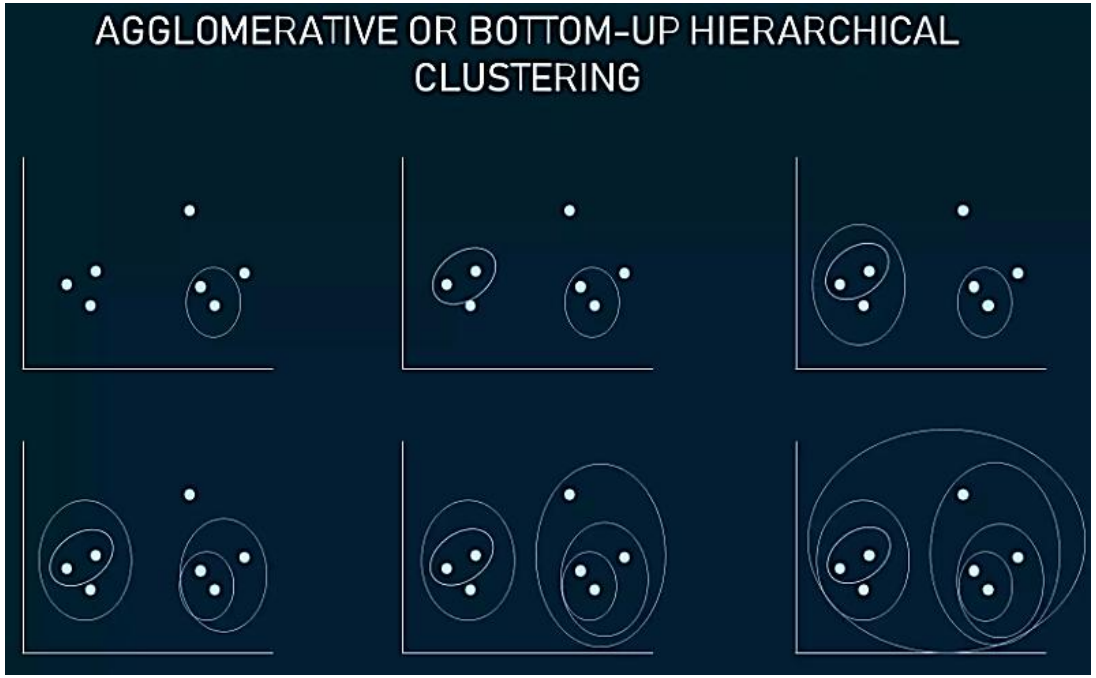
- **Exlcusive clustering**: or "hard" clustering
  - It is the kind of grouping in which one piece of data can belong only to one cluster.
- **Overlapping clustering**:
  - A soft cluster in which a single data point may belong to multiple clusters with varying degrees of membership.

Types of Unsupervised Learning

✓ **Clustering Algorithms**

– **Hierarchical Clustering:**

- Hierarchical clustering develops a hierarchy of clusters by merging or splitting them depending on their similarity. Here, two close cluster are going to be in the same cluster.
- In case you start with all data items attached to the same cluster and then **perform splits** until each data item is set as a separate cluster, the approach will be called **top-down or *divisive* hierarchical clustering.**
- Two clusters that are closest to one another are then **merged** into a single cluster. The merging goes on iteratively till there's only one cluster left at the top. Such an approach is known **as bottom-up or *agglomerative***.
- The example shows how seven different clusters (data points) are merged step by step based on distance until they all create one large cluster.

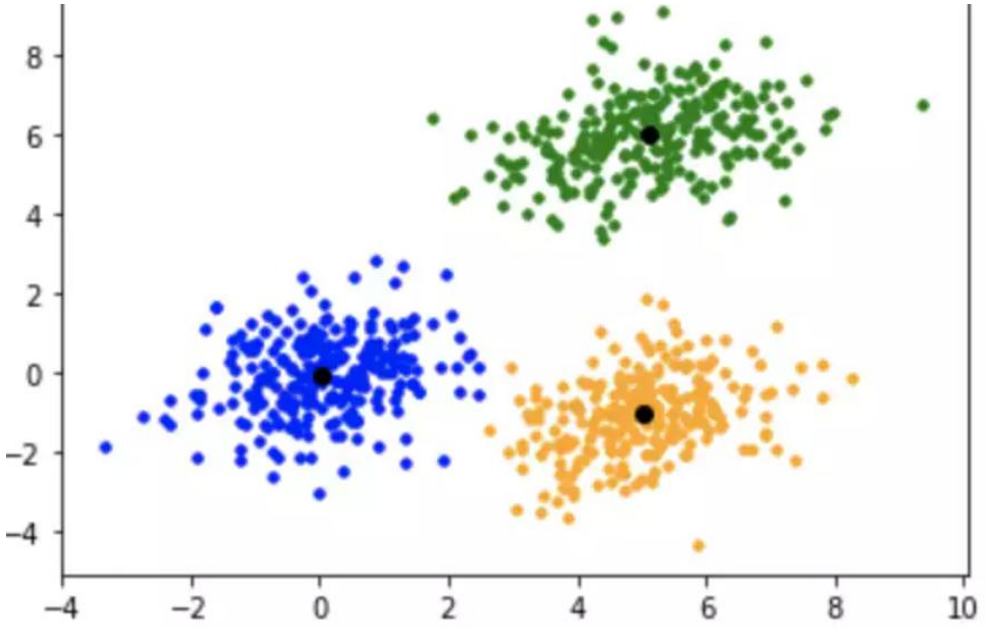


Types of Unsupervised Learning

✓ **Clustering Algorithms**

– **K-Means Clustering**

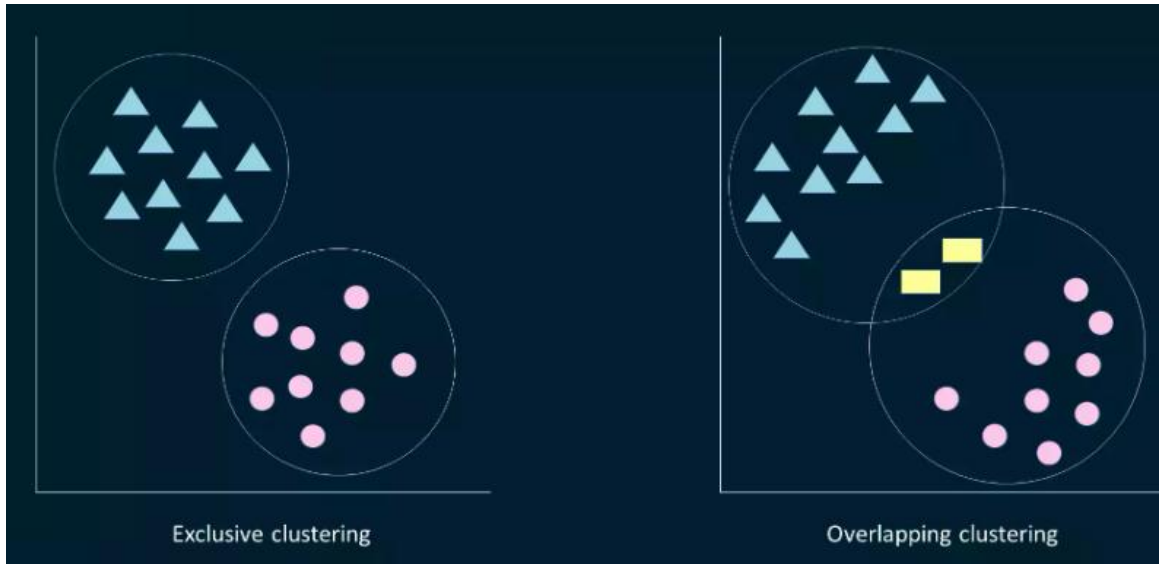
- In K-means clustering, data is grouped in terms of characteristics and similarities.
- K is a letter that represents the number of clusters. For example, if K=3, then the number of desired clusters is 3. If K=10, then the number of desired clusters is 10.
- It puts the data points into the predefined number of clusters *K*.
- Each data item then gets assigned to the nearest cluster center, called *centroids* (black dots in the picture). The latter act as data accumulation areas.
- The procedure of clustering may be repeated several times until the clusters are well-defined.



Ideal clustering with a single centroid in each cluster. Source: GeeksforGeeks

Types of Unsupervised Learning

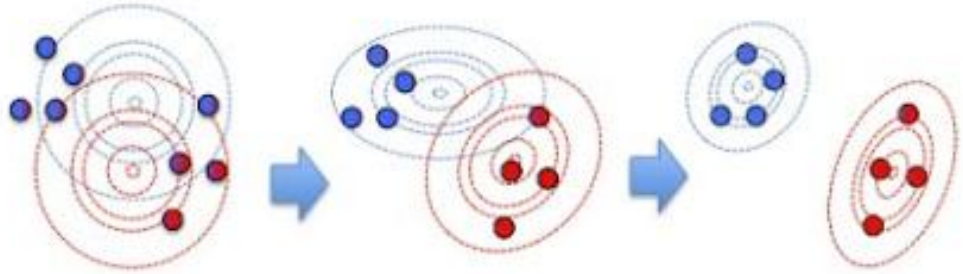
- ✓ **Clustering Algorithms**
  - **Fuzzy K-means**
    - It is an extension of the K-means algorithm used to perform overlapping clustering. Unlike the K-means algorithm, **fuzzy K-means implies that data points can belong to more than one cluster with a certain level of closeness towards each**.
    - The closeness is measured by the distance from a data point to the centroid of the cluster. So, sometimes there may be an overlap between different clusters.
  -



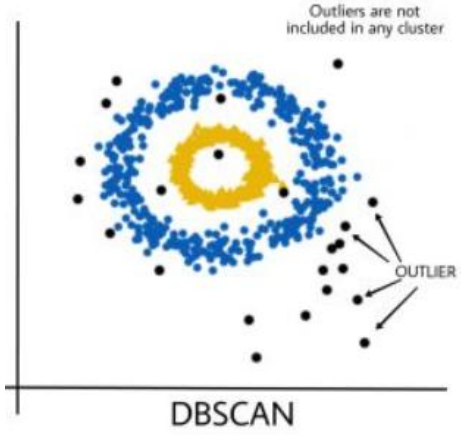
Types of Unsupervised Learning

✓ **Clustering Algorithms**

- **Gaussian Mixture Models** (GMMs) is an algorithm used in probabilistic clustering.
  - Models assume that there is a certain number of Gaussian distributions, each representing a separate cluster.
  - The algorithm is basically utilized to decide which cluster a particular data point belongs to.



- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** DBSCAN groups data points based on their density, identifying clusters of high-density regions and classifying outliers as noise.

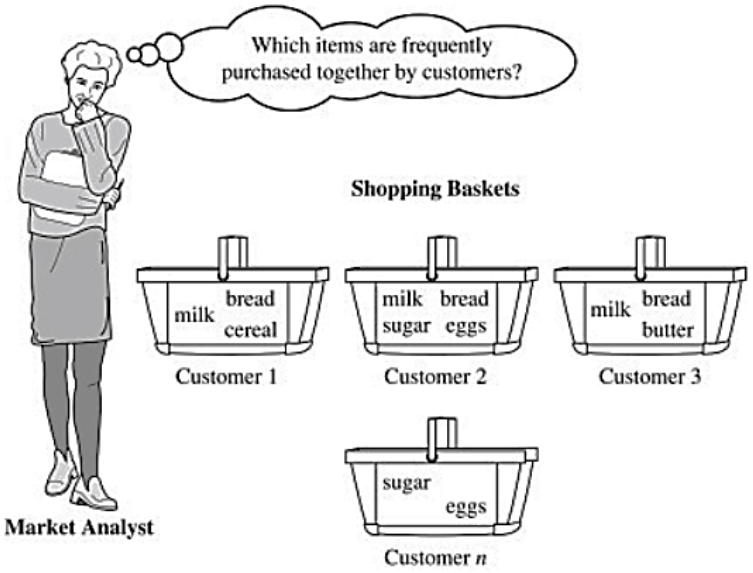


Types of Unsupervised Learning

✓ **Association Rule Mining**

– Association rule mining focuses on discovering interesting relationships or patterns between variables in the large database. It determines the **set of items that occurs together in the dataset**. Association rule makes marketing strategy more effective. Such as people who buy X (suppose a bread) also tend purchase Y (Butter/Jam) item. The widely used algorithm for association rule mining is the **Apriori** algorithm.

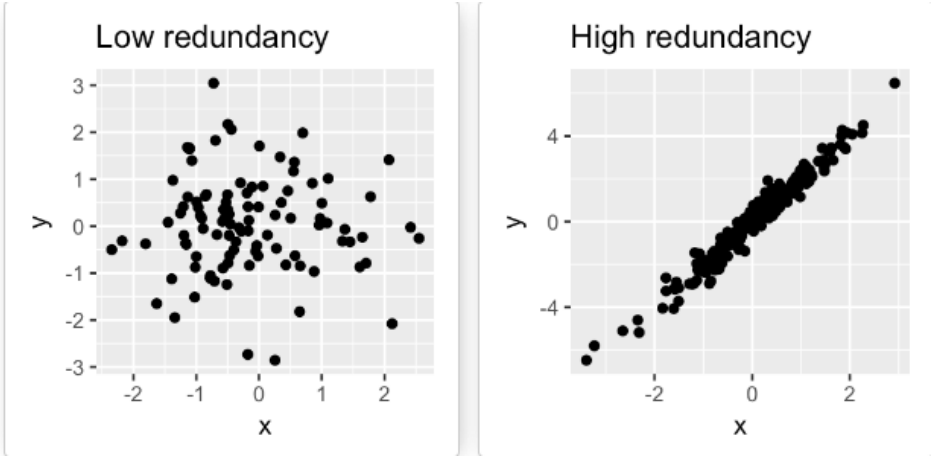
- **A real-life example of this is market basket analysis**, where retailers analyze customer purchase data to identify relationships between products frequently bought together. For instance, this analysis might reveal that customers who purchase diapers also tend to buy baby wipes



Types of Unsupervised Learning

✓ **Dimensionality Reduction Algorithms** Dimensionality reduction techniques are used to reduce the number of input variables or features while retaining meaningful information. Some popular dimensionality reduction algorithms include:

- **Principal Component Analysis (PCA):** PCA **transforms** the original features into a **lower-dimensional space** while **preserving** the maximum amount of **information**.
- The PCA method is particularly useful when the variables within the data set are highly correlated. Correlation indicates that there is redundancy in the data. Due to this redundancy, PCA can be used to reduce the original variables into a smaller number of new variables (**principal components**) explaining most of the variance in the original variables.

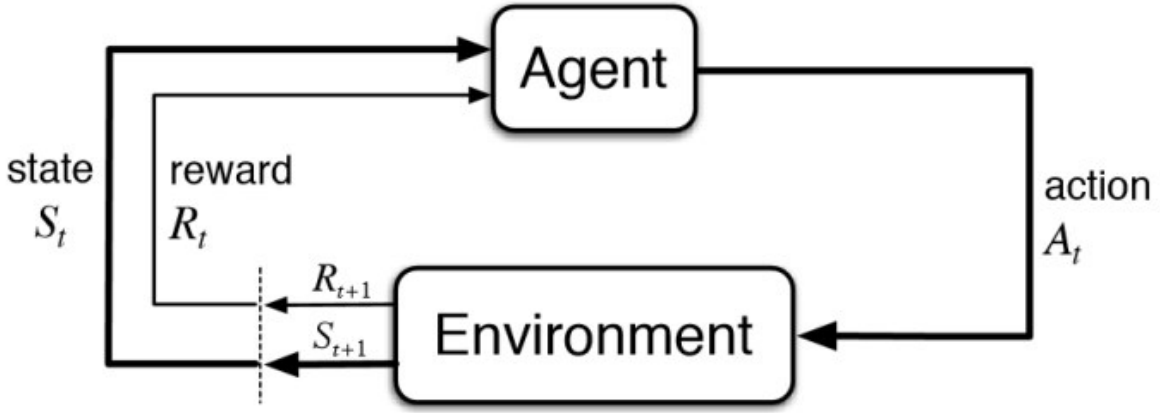


What is the difference between supervised and unsupervised learning?

- ✓ Supervised learning requires **labeled data** with **input features and corresponding output labels**, while unsupervised learning aims to **discover patterns or structures in unlabeled data** without predefined output labels.

Reinforcement machine learning

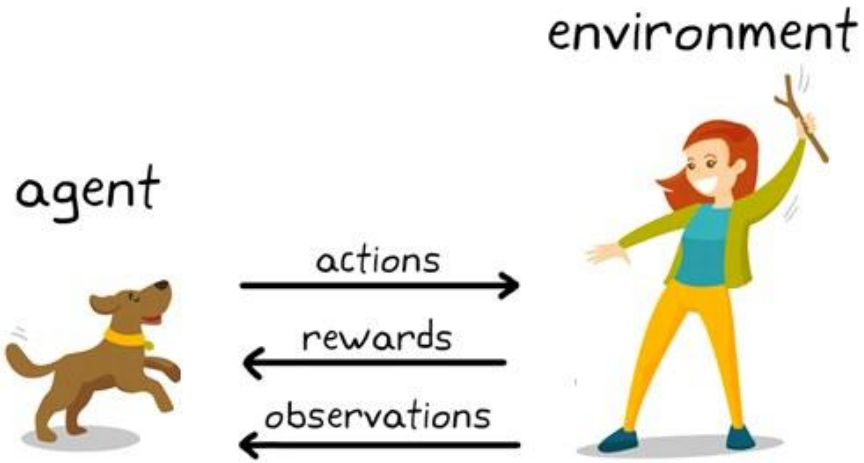
- ✓ Reinforcement Learning(RL) enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experiences.
- ✓ Here, agents are self-trained on reward and punishment mechanisms.
- ✓ It can take actions and interact with it.
- ✓ Reinforcement machine learning algorithm isn't trained using sample data.
- ✓ A sequence of successful outcomes will be reinforced to develop the best recommendation or policy for a given problem.



Basic Diagram of Reinforcement Learning

Reinforcement machine learning

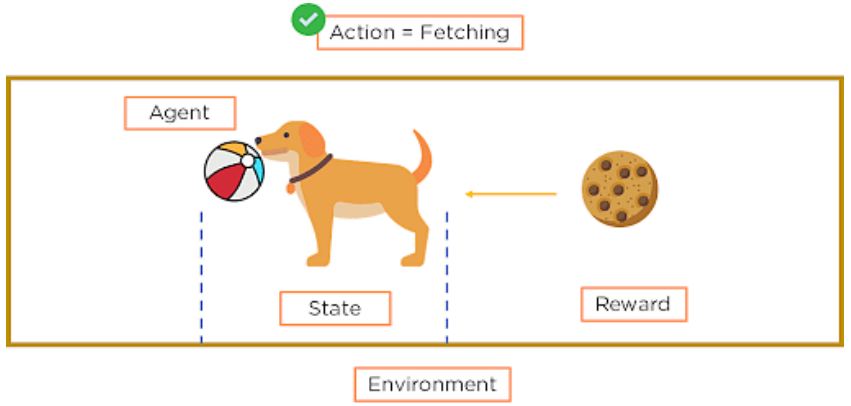
- ✓ Through a series of Trial and Error methods, an agent keeps learning continuously in an interactive environment from its own actions and experiences.
- ✓ The only goal of it is to find a suitable action model which would increase the total cumulative reward of the agent.
- ✓ It learns via interaction and feedback.
- ✓ You can see a dog and a master. Let's imagine you are training your dog to get the stick. Each time the dog gets a stick successfully, you offered him a feast (a bone).
- ✓ Eventually, the dog understands the pattern, that whenever the master throws a stick, it should get it as early as it can gain a reward (a bone) from a master in a lesser time.



Reinforcement Learning Example

Important Terms in Reinforcement Learning

- ✓ **Agent**: Agent is the model that is being trained via reinforcement learning. It is the sole decision-maker and learner
- ✓ **Environment**: a physical world where an agent learns and decides the actions to be performed
- ✓ **Action**: All possible steps that can be taken by the model/agent
- ✓ **State**: The current position/ condition returned by the model/the current situation of the agent in the environment
- ✓ **Reward**: To help the model move in the right direction, it is rewarded/points are given to it to appraise some action. **It's usually a scalar value and nothing but feedback from the environment**
- ✓ **Policy**: Policy determines how an agent will behave at any time. It acts as a mapping between Action and present State. The agent prepares strategy(decision-making) to map situations to actions.
- ✓ Value — Future reward that an agent would receive by taking an action in a particular state



SUPERVISED OR UNSUPERVISED?

SCENARIO - 1

Facebook Face Recognition

SCENARIO - 2

Netflix Movie Recommendation

SCENARIO - 3

Fraud Detection

Explanation

- ✓ **Scenario 1:** Facebook recognizes your friend in a picture from an album of tagged photographs
  - Explanation: It is supervised learning. Here Facebook is using tagged photos to recognize the person. Therefore, the tagged photos become the labels of the pictures and we know that when the machine is learning from labeled data, it is supervised learning.
- ✓ **Scenario 2:** Recommending new songs based on someone's past music choices
  - Explanation: It is supervised learning. The model is training a classifier on pre-existing labels (genres of songs). This is what Netflix, Pandora, and Spotify do all the time, they collect the songs/movies that you like already, evaluate the features based on your likes/dislikes and then recommend new movies/songs based on similar features.
- ✓ **Scenario 3:** Analyze bank data for suspicious looking transactions and flag the fraud transactions
  - Explanation: It is unsupervised learning. In this case, the suspicious transactions are not defined, hence there are no labels of "fraud" and "not fraud". The model tries to identify outliers by looking at anomalous transactions and **flags them as 'fraud'**.

Outline

✓ Linear Regression
- Linear models
- **Hypothesis function for Linear Regression**
- **Cost function**

Linear Regression

Linear regression is a statistical method that is used in various machine learning models to predict the value of unknown data using other related data values. Linear regression is used to study the relationship **between a dependent variable and an independent variable.**

Linear Regression Equation

$$Y = a + bx$$

$$a = \frac{[(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

$$b = \frac{[n(\Sigma xy) - (\Sigma x)(\Sigma y)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

Definition 2.3.1 (Linear Regression): Suppose we have an input $\mathbf{x} \in \mathbb{R}^D$ and a continuous target $y \in \mathbb{R}$. Linear regression determines weights $w_i \in \mathbb{R}$ that combine the values of x_i to produce y:

$$y = w_0 + w_1 x_1 + ... + w_D x_D \tag{2.1}$$

$\star$ Notice w_0 in the expression above, which doesn't have a corresponding x_0 value. This is known as the *bias* term. If you consider the definition of a line $y = mx + b$, the bias term is corresponds to the intercept b. It accounts for data that has a non-zero mean.

Let's illustrate how linear regression works using an example, considering the case of 10 year old Sam. She is curious about how tall she will be when she grows up. She has a data set of parents' heights and the final heights of their children

$$x_1 = \text{height of mother (cm)}$$
$$x_2 = \text{height of father (cm)}$$

Using linear regression, she determines the weights $\mathbf{w}$ to be:

$$\mathbf{w} = [34, 0.39, 0.33]$$

Sam's mother is 165 cm tall and her father is 185 cm tall. Using the results of the linear regression solution, Sam solves for her expected height:

$$\text{Sam's height} = 34 + 0.39(165) + 0.33(185) = \mathbf{159.4\ cm}$$

Linear Regression Equation

Linear regression line equation is written in the form: **y = a + bx**

where,

→ x is Independent Variable, Plotted along X-axis

→ y is Dependent Variable, Plotted along Y-axis

The slope of the regression line is “b”, and the intercept value of regression line is “a”(the value of y when x = 0).

Linear Regression Formula

Formula used for linear regressions is, **y = a + bx**

Intercept value, a, and slope of the line, b, are evaluated using the formulas given below:

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

where,

- y is Dependent Variable that Lies along Y-axis
- a is *Y-Intercept*
- b is *Slope of regression line*
- x is Independent Variable that Lies along X-axis

Properties of Linear Regression

In the linear regression line if the regression parameters a_0 and a_1 are defined, the properties are given as below:

→ Linear regression line reduces the sum of squared differences between observed values and predicted values.

→ Linear regression line always passes through the mean of X and Y variable
values.

→ Linear regression constant (b_0) is equal to the y-intercept of the linear regression.

→ Linear regression coefficient (b_0) is the slope of the regression line.

Regression Coefficient

Linear regression line, equation:

$Y = B_0 + B_1X$

where,

B_0 is a Constant

B_1 is Regression Coefficient

Here, B_1 is the regression coefficient and its formula is:

$$B_1 = b_1 = \Sigma [(x_i - x)(y_i - y)] / \Sigma [(x_i - x)^2]$$

where,

→ x_i and y_i are Observed Data Sets

→ x and y are Mean Value

What is Linear Regression Used for?

Various uses of Linear Regression are:

→ It is used in market research and study of customer survey results.

→ It is used for studying performance of engine of automobiles.

→ It is used in deciding the effective price of any goods.

→ It is used in astronomy.

Regression Coefficient

Linear regression line, equation:

$Y = B_0 + B_1X$

where,

B_0 is a Constant

B_1 is Regression Coefficient

Here, B_1 is the regression coefficient and its formula is,

$$B_1 = b_1 = \Sigma [(x_i - x)(y_i - y)] / \Sigma [(x_i - x)^2]$$

where,

→ x_i and y_i are Observed Data Sets

→ x and y are Mean Value

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

Question 1: Find the linear regression equation for the given data:

x	y
3	8
9	6
5	4
3	2

Calculating intercept and slope value.

x	y	x^2	xy
3	8	9	24
9	6	81	54
5	4	25	20
3	2	9	6
$\sum x = 20$	$\sum y = 20$	$\sum x^2 = 124$	$\sum xy = 104$

Using formula for *a*.

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

x	y	x^2	xy
3	8	9	24
9	6	81	54
5	4	25	20
3	2	9	6
$\sum x = 20$	$\sum y = 20$	$\sum x^2 = 124$	$\sum xy = 104$

a = {20 (124) – 20 (104)} / {4 (124) – 400}

a = 400/96 = 4.17

Using formula for *b*.

$$b = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

b = {4 (104) – 20 (20)} / {4 (124) – 400}

b = 16/96 = 0.166

So, linear regression equation is:

y=a+bx => y = 4.17 + 0.166x

Find the intercept of linear regression line if $\sum x = 25$, $\sum y = 20$, $\sum x^2 = 90$, $\sum xy = 150$ and n = 5.

Using formula,

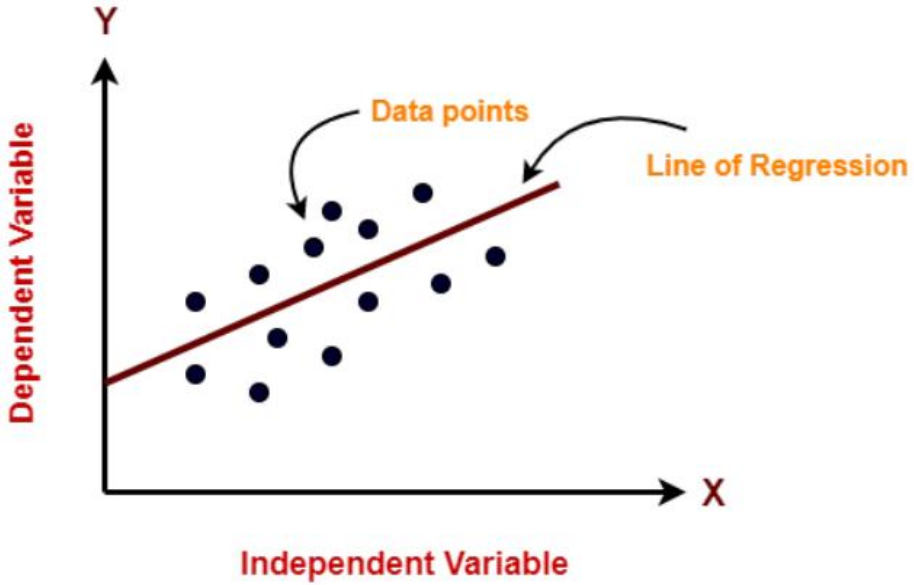
$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

= (20 (90) – 25 (150)) / (5 (90) – 625)

= -1950/-175

= 11.14

SLOPE??



The equation for a simple linear regression model can be written as follows:

$$y = b_0 + b_1 * x$$

Here, **y is the dependent variable (the variable we are trying to predict)**, **x is the independent variable (the predictor or feature)**.

b0 is the intercept term (the value of y when x is zero), and

b1 is the slope coefficient (the change in y for a unit change in x).

The goal of Linear Regression is to find the best values for b0 and b1 such that the line best fits the data points, minimizing the errors or the difference between the predicted values and the actual values.

Types of Linear Regression?

There are two main types of Linear Regression models:

-- **Simple Linear Regression** and

-- Multiple Linear Regression.

Simple Linear Regression: In simple linear regression, there is only one independent variable (also known as the predictor or feature) and one dependent variable (also known as the response variable). The goal of simple linear regression is to find the best-fitting line to describe the relationship between the independent and dependent variable. The equation for a simple linear regression model can be written as:

$$Y = b_0 + b_1 * X$$

Here, Y is the dependent variable, X is the independent variable, b0 is the intercept term, and b1 is the slope coefficient.

Multiple Linear Regression: In multiple linear regression, there are multiple independent variables and one dependent variable. The goal of multiple linear regression is to find the best-fitting line to describe the relationship between the independent variables and the dependent variable. The equation for a multiple linear regression model can be written as:

$$Y = b_0 + b_1 * X_1 + b_2 * X_2 + \ldots + b_n * X_n$$

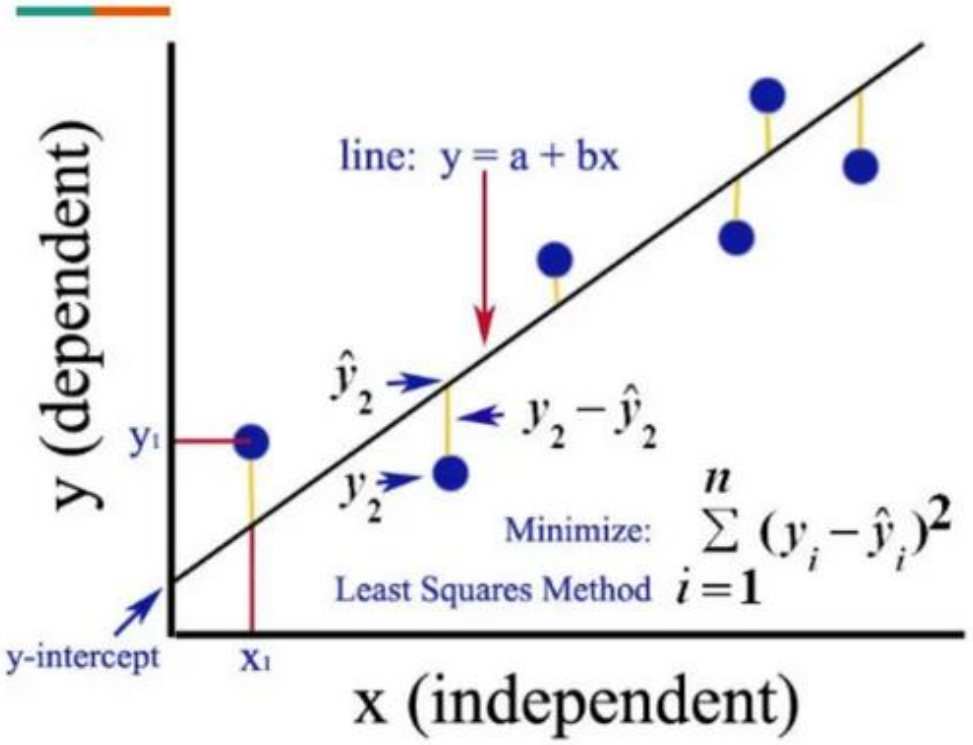
Here, Y is the dependent variable, X1, X2, …,
Xn are the independent variables,
b0 is the intercept term, and
b1, b2, …, bn are the slope coefficients.

In both types of linear regression, the goal is to find the best values for the intercept and slope coefficients that minimize the difference between the predicted values and the actual values.

Finding the best fit line

In machine learning, finding the best-fitting line is crucial in linear regression, as it determines the accuracy of the predictions made by the model. *The best-fitting line is the line that has the smallest difference between the predicted values and the actual values*.

To find the best-fitting line in a linear regression model, one uses a process called "**ordinary least squares (OLS) regression**". This process involves calculating the sum of the squared differences between the predicted values and the actual values for each data point, and then finding the line that minimizes this sum of squared errors.



The best-fitting line is found by minimizing the residual sum of squares (RSS), *which is the sum of the squared differences between the predicted values and the actual values*.

This is achieved by adjusting the values of the intercept and slope coefficients, also known as c and m, respectively.

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

Once the values of c and m are determined, we can use the linear regression equation to make predictions for new data points. The equation for a simple linear regression model can be written as:

y = c + m * x

Here, y is the dependent variable (the variable we are trying to predict), x is the independent variable (the predictor or feature), c is the intercept term (the value of y when x is zero), and m is the slope coefficient (the change in y for a unit change in x).

Cost function

The cost function, also known as the loss function or objective function, is a measure of how well the model is performing. In linear regression, the cost function is used to calculate the difference between the predicted values and the actual values, also known as the residuals or errors.

The goal of linear regression is to minimize the cost function, which is achieved by finding the best-fitting line that minimizes the sum of the squared differences between the predicted values and the actual values. The most commonly used cost function in linear regression is the **Mean Squared Error (MSE)**, which is calculated as the average of the squared differences between the predicted and $MSE = (1/n) * \Sigma(yi - \hat{y}i)^2$

* Here, n is the number of data points, y_i is the actual value, and $\hat{y}_i$ is the predicted value.

Model Performance

Model performance in linear regression can be evaluated using various metrics that measure how well the model fits the data and how well it generalizes to new data. Go to next slides for common metrics used in linear regression: ……

Example:

Let, there is a linear regression model that predicts housing prices based on the size of the house. Here, 10 data points with the following **actual prices** (y) and **predicted prices** (y_hat):

y = [100, 150, 200, 250, 300, 350, 400, 450, 500, 550]

y_hat = [110, 140, 180, 240, 290, 320, 380, 420, 480, 520]

*** y_mean = sum(y) / len(y) = 325**

1. Sum of Squares Regression (SSR):

SSR is calculated by taking the sum of the squared differences between the predicted values and the mean of the dependent variable. It represents the variability in the dependent variable that is explained by the independent variables. Mathematically, it can be expressed as:

$$SSR = sum((y\_hat - y\_mean)^2) = 28300$$

2. Sum of Squares Error (SSE):

SSE is calculated by taking the sum of the squared differences between the predicted values and the actual values of the dependent variable. Mathematically, it can be expressed as:

$$SSE = sum((y - y\_hat)^2) = 26700$$

3. Mean Squared Error (MSE):

This metric measures the average squared difference between the predicted and actual values. It is calculated as the sum of squared residuals divided by the number of data points. A lower MSE indicates better performance. Mathematically, it can be expressed as:

$$MSE = (1/n) * sum((y - y\_hat)^2) = 2670$$

4. Root Mean Squared Error (RMSE):

This metric measures the square root of the MSE and is useful for interpreting the errors in the same units as the dependent variable. Mathematically, it can be expressed as:

RMSE = sqrt(MSE) = sqrt(2670) = 51.68

5. Mean Absolute Error (MAE):

This metric measures the average absolute difference between the predicted and actual values. It is less sensitive to outliers than MSE, but still provides a measure of the model's accuracy. Mathematically, it can be expressed as:

MAE = (1/n) * sum(abs(y — y_hat)) = 36

6. Sum Of Squares Total (SST):

SST is calculated by taking the sum of the squared differences between the actual values of the dependent variable and its mean value. Mathematically, it can be expressed as:

SST = sum((y — y_mean)²) = 55000

R-Squared (R^2):
This metric measures the proportion of variance in the dependent variable that is explained by the independent variable(s).

It ranges from 0 to 1, with a higher value indicating better performance.

$$R^2 = SSR / SST = 28300 / 55000 = 0.5164$$
$$R^2 = 1 — SSE / SST = 1 —26700 / 55000 = 0.5164$$

Outline

✓ Linear Regression
  - **Gradient descent algorithm**
  - Polynomial regression

Gradient descent algorithm

Basic Idea

Suppose you are standing on a hill and want to reach the **lowest point**.

What will you do?

- Look around
- Find the downward direction
- Take a small step
- Repeat until you reach the bottom

Gradient Descent works exactly like this.

- The **hill** = Cost/Loss Function
- The **lowest point** = Minimum error
- The **steps** = Parameter updates

Why Do We Need Gradient Descent?

In Machine Learning, models make predictions using parameters.

Example of a linear model:

$$y = mx + b$$

Where:

- m = slope (weight)
- b = intercept (bias)

We need to find the best values of m and b so prediction error becomes minimum.

Gradient Descent helps us find those best values.

Gradient Descent Formula

Parameters are updated using:

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial J}{\partial \theta}$$

Where:

>> θ = parameter (weight/bias)

>> α = learning rate

>> $\frac{\partial J}{\partial \theta}$ = gradient (derivative)

Learning Rate

The learning rate controls step size.

Small Learning Rate

- Very slow learning
- Takes many iterations

Large Learning Rate

- May overshoot minimum
- Can fail to converge

Example:

- $\alpha = 0.01$
- $\alpha = 0.1$

1

Cost Function

The algorithm minimizes a **cost function**.

For Linear Regression, a common cost function is:

$$J(m,b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Where:

- $J(m,b)$ = cost function
- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of data points

This is called **Mean Squared Error (MSE)**.

2

Gradient Descent Formula

Parameters are updated using:

$$\theta_{new} = \theta_{old} - \alpha\frac{\partial J}{\partial \theta}$$

Where:

- θ = parameter (weight/bias)
- α = learning rate
- $\frac{\partial J}{\partial \theta}$ = gradient (derivative)

3

Step-by-Step Numerical Example

Suppose: $J(w) = w^2$

We want to minimize this function.

The minimum value occurs at:

$$w = 0$$

Step 1: Find Derivative

Derivative:

$$\frac{dJ}{dw} = 2w$$

Step 2: Initialize Value

Assume:

- Initial $w = 8$
- Learning rate $\alpha = 0.1$

Update rule:

$$w_{new} = w_{old} - 0.1(2w)$$

Iteration 1

Current value:

$$w = 8$$

Gradient:

$$2(8) = 16$$

Update:

$$w = 8 - 0.1(16)$$

$$w = 8 - 1.6$$

$$w = 6.4$$

4

We continue until w becomes close to 0.

Iteration 2

Gradient:

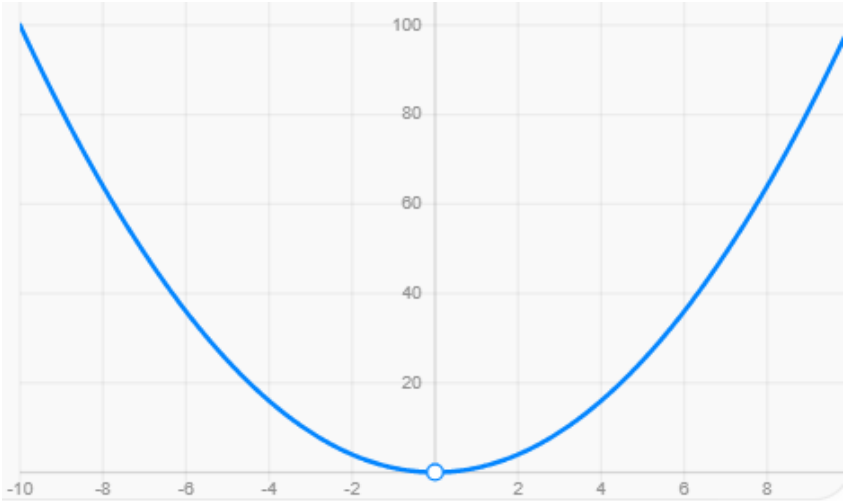
$$2(6.4) = 12.8$$

Update:

$$w = 6.4 - 0.1(12.8)$$

$$w = 5.12$$

5



Application of Gradient Descent in Linear Regression

Problem Statement

Suppose we have this dataset:

Hours Studied (x)	Marks (y)
1	2
2	4
3	6

We want to build a model that predicts marks from study hours.

Linear Regression Model

Linear Regression uses this equation:

$$\hat{y} = wx + b$$

Where:

- $\hat{y}$ = predicted output
- x = input feature
- w = weight (slope)
- b = bias/intercept

Goal:

- Find the best values of w and b

Initialize Parameters

Initially we start with random values.

Assume:

$$w = 0$$

$$b = 0$$

So the model becomes:

$$\hat{y} = wx + b$$

$$\hat{y} = 0x + 0$$

$$\hat{y} = 0$$

This means:

The model predicts 0 for every input.

Clearly wrong.

Data Point 1

Input: $x = 1$

Prediction: $\hat{y} = 0(1) + 0 = 0$

Actual value: **from given table**

$$y = 2$$

Error:

$$0 - 2 = -2$$

Data Point 2

$$x = 2$$

Prediction: $\hat{y} = 0(2) + 0 = 0$

Actual: $y = 4$

Error: $0 - 4 = -4$

Data Point 3

$$x = 3$$

Prediction: $\hat{y} = 0(3) + 0 = 0$

Actual: $y = 6$

Error: $0 - 6 = -6$

Calculate Cost Function

We use Mean Squared Error:

$$J(w,b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Here:

- $n = 3$ **and**

Errors:

- -2
- -4
- -6

Square them:

$$(-2)^2 = 4$$

$$(-4)^2 = 16$$

$$(-6)^2 = 36$$

Add: $4 + 16 + 36 = 56$

Divide by 3: **[because n = 3]**

$$J(w,b) = \frac{56}{3}$$

$$J(w,b) = 18.67$$

This error is very high.

Goal:

Reduce this cost.

How Gradient Descent Helps

Gradient Descent updates:

- w
- b

so the cost decreases.

Derivatives (Gradients)

For Linear Regression:

Gradient for w:

$$\frac{\partial J}{\partial w} = \frac{2}{n}\sum x_i(\hat{y}_i - y_i)$$

Gradient for b:

$$\frac{\partial J}{\partial b} = \frac{2}{n}\sum(\hat{y}_i - y_i)$$

These tell us:

- how much to change w
- how much to change b

Calculate Gradients

Currently:

- $w = 0$
- $b = 0$

Predictions:

- $0, 0, 0$

Errors:

- $-2, -4, -6$

Problem Statement

Suppose we have this dataset:

Hours Studied (x)	Marks (y)
1	2
2	4
3	6

We want to build a model that predicts marks from study hours.

Gradient for w

Formula:

$$\frac{2}{n}\sum x_i(\hat{y}_i - y_i)$$

Substitute values:

$$= \frac{2}{3}[(1)(-2) + (2)(-4) + (3)(-6)]$$

Calculate:

$$= \frac{2}{3}[-2 - 8 - 18]$$

$$= \frac{2}{3}[-28]$$

$$= -18.67$$

Gradient for b

$$= \frac{2}{3}[(-2) + (-4) + (-6)]$$

$$= \frac{2}{3}[-12]$$

$$= -8$$

Update Parameters

Gradient Descent update rule:

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J}{\partial b}$$

Assume learning rate: $\alpha = 0.1$

Update w

$$w = 0 - 0.1(-18.67)$$

$$w = 1.867$$

Update b

$$b = 0 - 0.1(-8)$$

$$b = 0.8$$

New Model

Now model becomes:

$$\hat{y} = 1.867x + 0.8$$

New Predictions

For $x = 1$

$$\hat{y} = 1.867(1) + 0.8$$

$$= 2.667$$

Actual: 2

Error smaller now.

For $x = 2$

$$\hat{y} = 1.867(2) + 0.8$$

$$= 4.534$$

Actual: 4

Again better.

For $x = 3$

$$\hat{y} = 1.867(3) + 0.8$$

$$= 6.401$$

Actual: 6

Much closer.

Repeat Again and Again

Gradient Descent repeats:

1. Predict
2. Calculate error
3. Calculate gradients
4. Update parameters

until:

- error becomes very small
- model converges

What is Actually Happening?

Initially:

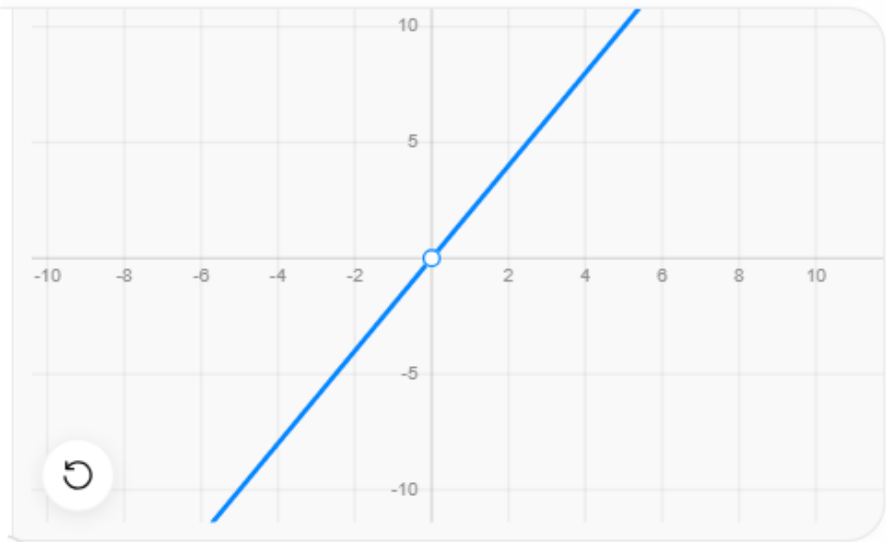
- line was completely wrong

After updates:

- line rotates and shifts
- gets closer to data points

Eventually it may become:

$$y = 2x$$



which perfectly fits this dataset.

```
import numpy as np

# Dataset
x = np.array([1,2,3])
y = np.array([2,4,6])

# Initial values
w = 0
b = 0

# Learning rate
alpha = 0.1
```

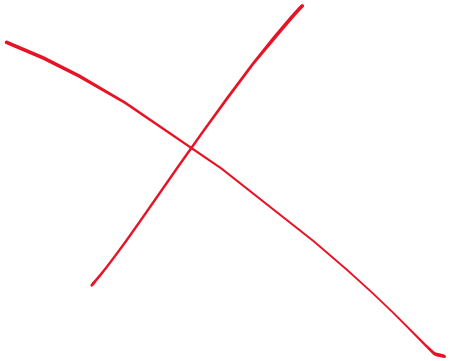
```
# Training
for i in range(5):
    y_pred = w*x + b
    error = y_pred - y

    dw = (2/len(x)) * np.sum(x * error)
    db = (2/len(x)) * np.sum(error)

    w = w - alpha * dw
    b = b - alpha * db

    cost = np.mean(error**2)

    print("Iteration:", i+1)
    print("w =", w)
    print("b =", b)
    print("cost =", cost)
    print()
```

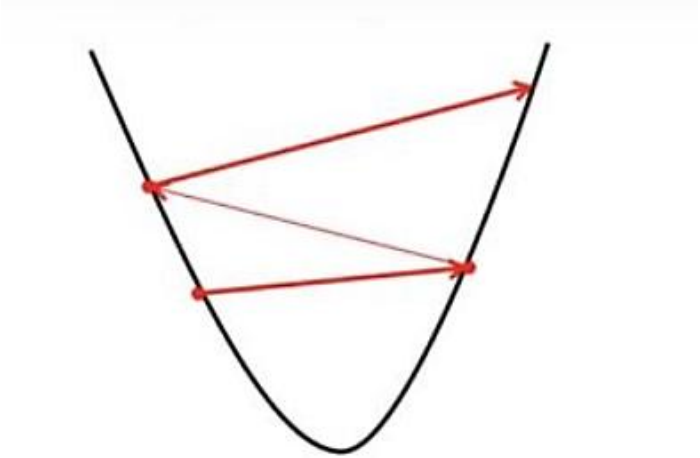


Why Derivatives?

Intuition of w and b

How To Choose Learning Rate

- ✓ The choice of correct learning rate is very important as it ensures that Gradient Descent converges in a reasonable time.
- ✓ If we choose **α to be very large**, Gradient Descent can overshoot the minimum. It may fail to converge or even diverge.
- ✓ If we choose **α to be very small**, Gradient Descent will take small steps to reach local minima and will take a longer time to reach minima.

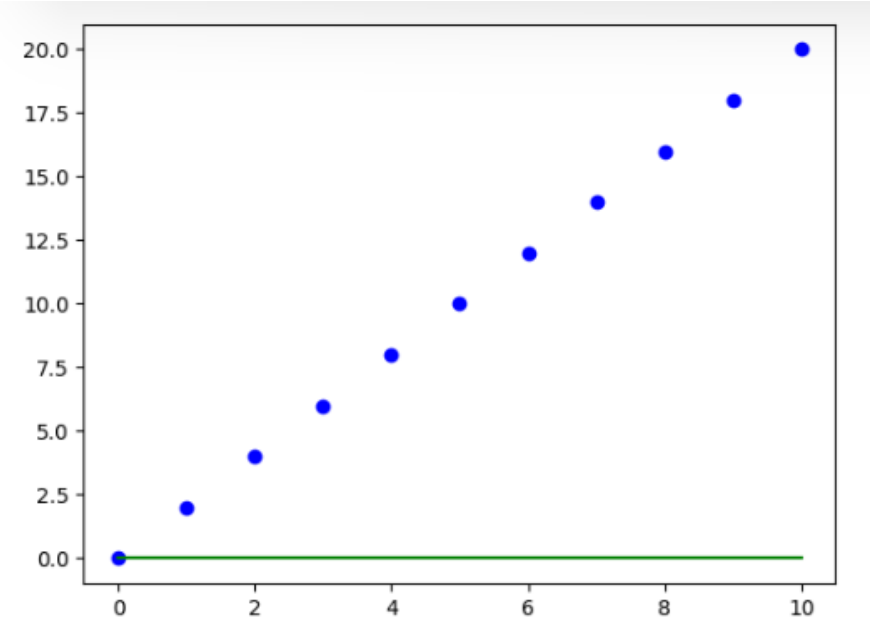


Effect of large alpha value on Gradient Descent

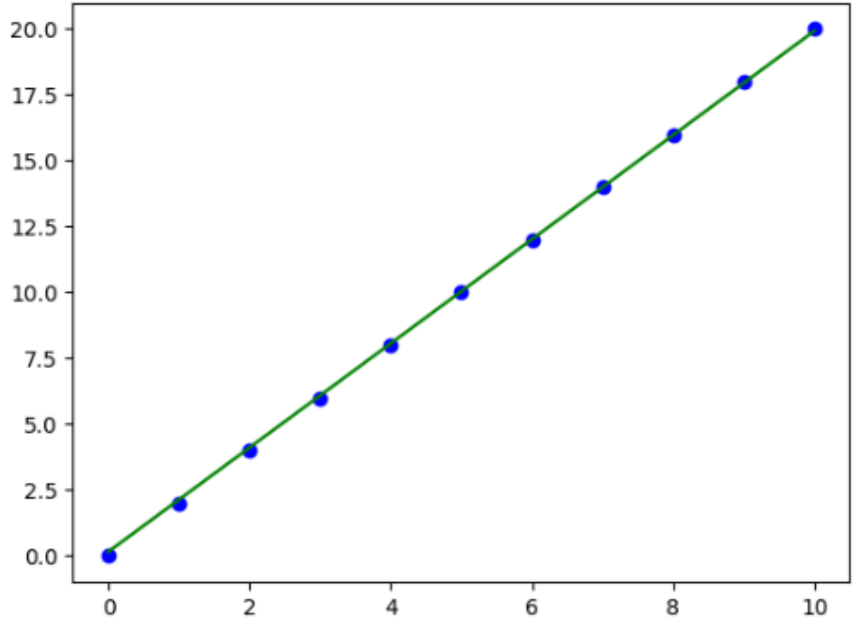
Effect of small alpha value on Gradient Descent

Example in a Real-World Scenario

- ✓ Consider a machine learning model where you're trying to minimize the error between predicted and actual values (the loss function). The parameters could be the weights in a linear regression model_predicting house prices. The gradient descent algorithm would iteratively adjust these weights to minimize the difference (loss) between the predicted and actual house prices.



Regression line before gradient descent iteration



Regression line after gradient descent iteration

Polynomial Regression (Beginner Friendly Detailed Explanation)

Polynomial Regression is a type of regression used when the relationship between input and output is **not linear**.

It is an extension of **Linear Regression**.

Why Do We Need Polynomial Regression?

Suppose we want to predict:

- student performance
- population growth
- stock trends
- temperature variation
- disease spread

Sometimes data does NOT follow a straight line.

Example of Non-Linear Relationship

Age	Running Speed
5	10
10	18
15	25
20	28
25	24
30	18

Here:

- speed increases first
- then decreases

This is NOT a straight line.

Linear Regression fails here.

What is Polynomial Regression?

Polynomial Regression fits a **curve** instead of a straight line.

General form:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \cdots + b_nx^n$$

Where:

- x = input
- x^2, x^3 = polynomial terms
- b_0, b_1, b_2 = coefficients

Difference Between Linear and Polynomial Regression

Linear Regression	Polynomial Regression
Straight line	Curved line
Simple relationship	Complex relationship
Equation: $y = mx + b$	Equation contains x^2, x^3 etc.

Degree 1

$y = b_0 + b_1x$

This is Linear Regression.

Degree 2 (Quadratic)

$y = b_0 + b_1x + b_2x^2$

Produces parabola shape.

Degree 3 (Cubic)

$y = b_0 + b_1x + b_2x^2 + b_3x^3$

More flexible curve.

Simple Numerical Example

Suppose dataset:

x	y
1	1
2	4
3	9
4	16

Observe carefully: $y = x^2$

because: $1^2 = 1$

$2^2 = 4$

$3^2 = 9$

$4^2 = 16$

A straight line cannot fit these points properly.

But Polynomial Regression can.

Model Building

Suppose model:

$$y = b_0 + b_1 x + b_2 x^2$$

Assume:

- $b_0 = 0$
- $b_1 = 0$
- $b_2 = 1$

Then: $y = x^2$

Prediction Example

For: $x = 5$

Prediction:

$$y = 5^2$$

$$y = 25$$

Why Polynomial Regression is Still "Linear"

This confuses many beginners.

Even though equation contains:

- x^2
- x^3

it is still linear in parameters.

Example: $y = b_0 + b_1 x + b_2 x^2$

Parameters:

- b_0, b_1, b_2

They are NOT squared.

So training still uses:

- Linear Regression techniques
- Gradient Descent

Cost Function

Polynomial Regression often uses Mean Squared Error:

$$J = \frac{1}{n} \sum (\hat{y} - y)^2$$

Goal:

- minimize error

using Gradient Descent.

Gradient Descent in Polynomial Regression

Suppose model: $\hat{y} = b_0 + b_1 x + b_2 x^2$

Gradient Descent updates:

- b_0
- b_1
- b_2

again and again.

Student Exam Score based on Study Hours

Study Hours (x)	Exam Score (y)
1	3
2	8
3	15
4	24
5	35

Observe carefully:

The score is increasing faster over time.

A straight line will not fit properly.

So we use: **Polynomial Regression**

Observe the Pattern

Check relationship manually.

For $x = 1$:

$$1^2 + 2(1) = 3$$

For $x = 2$:

$$2^2 + 2(2) = 8$$

For $x = 3$:

$$3^2 + 2(3) = 15$$

Polynomial Regression Model

General quadratic polynomial model

$$y = b_0 + b_1 x + b_2 x^2$$

Where:

- b_0 = intercept
- b_1 = coefficient of x
- b_2 = coefficient of x^2

Find Coefficients

From the pattern: $y = x^2 + 2x$

Comparing with: $y = b_0 + b_1 x + b_2 x^2$

we get:

$$b_0 = 0$$

$$b_1 = 2$$

$$b_2 = 1$$

Example Prediction

Suppose student studies: $x = 6$ hours

Prediction: $y = 6^2 + 2(6)$

$= 36 + 12$

$= 48$

Predicted Score = 48

An AI system predicts salary growth based on years of experience.

Experience (x)	Salary (y)
1	6
2	14
3	24
4	36

Tasks:

1. Identify polynomial relationship
2. Write polynomial equation
3. Predict salary for $x = 5$

Study Hours (x)	Score (y)
1	6
2	11
3	18

We observe:

- scores are increasing non-linearly
- relationship is curved

Choose Polynomial Model

We choose degree 2 polynomial:

$$\hat{y} = b_0 + b_1 x + b_2 x^2$$

Where:

- b_0 = intercept
- b_1 = coefficient of x
- b_2 = coefficient of x^2

Goal:

Find best values of b_0, b_1, b_2

using Gradient Descent.

Initialize Parameters

Initially assume:

$$b_0 = 0$$

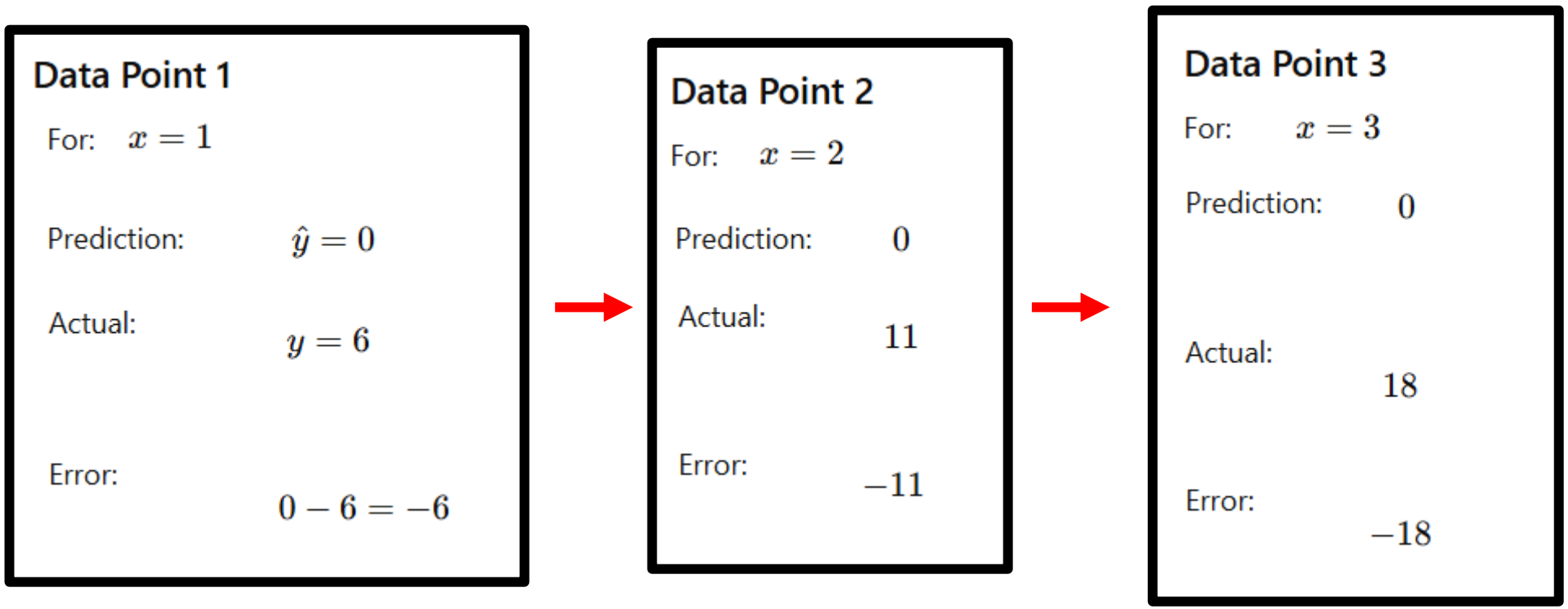
$$b_1 = 0$$

$$b_2 = 0$$

So initial model:

$$\hat{y} = 0$$

All predictions become 0.


Data Point 1
For: $x = 1$
Prediction: $\hat{y} = 0$
Actual: $y = 6$
Error: $0 - 6 = -6$
Data Point 2
For: $x = 2$
Prediction: 0
Actual: 11
Error: -11
Data Point 3
For: $x = 3$
Prediction: 0
Actual: 18
Error: -18

Cost Function

We use Mean Squared Error:

$$J = \frac{1}{n}\sum(\hat{y} - y)^2$$

Errors:

- -6
- -11
- -18

Square them:

$$36,\ 121,\ 324$$

Add:

$$36 + 121 + 324 = 481$$

Divide by $n = 3$:

$$J = \frac{481}{3}$$

$$J = 160.33$$

Huge error.

Goal: **Minimize this cost.**

Compute Gradients

We need derivatives for:

- b_0
- b_1
- b_2

Gradient for b_0

$$\frac{\partial J}{\partial b_0} = \frac{2}{n}\sum(\hat{y} - y)$$

Substitute values:

$$= \frac{2}{3}[(-6) + (-11) + (-18)]$$

$$= \frac{2}{3}(-35)$$

$$= -23.33$$

Gradient for b_1

$\frac{\partial J}{\partial b_1} = \frac{2}{n}\sum x(\hat{y} - y)$

Substitute:

$$= \frac{2}{3}[(1)(-6) + (2)(-11) + (3)(-18)]$$

$$= \frac{2}{3}[-6 - 22 - 54]$$

$$= \frac{2}{3}(-82)$$

$$= -54.67$$

Gradient for b_2

$\frac{\partial J}{\partial b_2} = \frac{2}{n}\sum x^2(\hat{y} - y)$

Substitute:

$$= \frac{2}{3}[(1^2)(-6) + (2^2)(-11) + (3^2)(-18)]$$

$$= \frac{2}{3}[-6 - 44 - 162]$$

$$= \frac{2}{3}(-212)$$

$$= -141.33$$

Apply Gradient Descent

Gradient Descent update rule:

$b_{new} = b_{old} - \alpha\frac{\partial J}{\partial b}$

Assume learning rate:

$$\alpha = 0.01$$

Update b_0

$$b_0 = 0 - 0.01(-23.33)$$

$$b_0 = 0.2333$$

Update b_1

$$b_1 = 0 - 0.01(-54.67)$$

$$b_1 = 0.5467$$

Update b_2

$$b_2 = 0 - 0.01(-141.33)$$

$$b_2 = 1.4133$$

New Polynomial Model

After one iteration:

$$\hat{y} = 0.2333 + 0.5467x + 1.4133x^2$$

Now model is much better.

New Predictions

For $x = 1$

$$\hat{y} = 0.2333 + 0.5467(1) + 1.4133(1)^2$$

$$= 2.1933$$

Closer to actual value 6.

For $x = 2$

$$\hat{y} = 0.2333 + 0.5467(2) + 1.4133(4)$$

$$= 6.98$$

Closer to 11.

For $x = 3$

$$\hat{y} = 0.2333 + 0.5467(3) + 1.4133(9)$$

$$= 14.59$$

Closer to 18.

Repeat Iterations

Gradient Descent repeats:

1. Predict
2. Calculate error
3. Compute gradients
4. Update coefficients

again and again.

Eventually model converges near true relationship.

Thank You